BREM — Build Reproducibility & Environment Module

BREM — Build Reproducibility & Environment Module

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Configuration Standard (SCFS)

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Type: Core Module

Governed Space: Software Build Reproducibility & Environment Governance

Module ID: SLGA-SCFS-BREM-005

Origin ID: OOF.CM.SLGA.04.05.01

Version: 1.0

Status: Canonical · Open Standard

Effective Date: 12.10.2026

Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: MIP® — Methodological Intellectual Property

Canonical Language: English (UCL™)

Canonical Definition

The Build Reproducibility & Environment Module (BREM) governs the

identification, specification, preservation, comparison, reconstruction, and validation of software build environments and the conditions under which software build results can be meaningfully reproduced.

BREM establishes the methodological conditions under which a software build

can be recognized as reproducible within a defined scope, using sufficiently identified inputs, toolchains, parameters, environmental conditions, execution dependencies, and competent output-comparison evidence.

It governs the relationship between an original build, its effective

execution environment, relevant configuration and artifact references, the conditions required for reconstruction, a subsequent reproduction attempt, observed environmental differences, and the correspondence between original and reproduced outputs.

BREM distinguishes recorded build environments from verified execution

environments, repeated build execution from demonstrated reproducibility, identical source code from equivalent build conditions, and technically successful reproduction attempts from evidence-supported reproduction claims.

It applies to software builds performed by humans, organizations, automated

pipelines, continuous integration systems, AI coding assistants, autonomous development agents, and software infrastructure mechanisms.

Its principal output is:

Governed Build Reproducibility & Environment State

Its foundational distinction is:

Repeated Build Execution ≠ Demonstrated Build Reproducibility.

Operational Role

The operational role of BREM is:

To ensure that material software build environments and reproducibility claims remain identifiable, bounded, evidence-supported, and verifiable through competent comparison of build inputs, execution conditions, environmental dependencies, and resulting artifacts without confusing successful repeated execution with demonstrated reproducibility or software correctness.

BREM provides the build reproducibility and environment-governance

mechanism of SCFS.

It establishes whether a build's relevant execution conditions can be sufficiently identified, preserved, reconstructed, and compared with those of a subsequent build.

BREM does not independently authorize software creation, execute the complete build-management process, certify artifact provenance, approve configuration changes, validate software quality, or authorize release and deployment.

It provides an evidence-supported reproducibility and environment state for

competent SCFS and SLGA™ governance.

Operational Question

Under which identifiable and verifiable environmental conditions can a software build be reconstructed, and what evidence demonstrates that a subsequent build reproduces the original result within a clearly defined scope and comparison criterion?

Position in Architecture Governance Flow

The SLGA™ governance flow is:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting:

Software Lifecycle Control

BREM operates within:

Software Configuration Standard (SCFS)

The SCFS internal governance flow is:

Configuration Baseline → Build Management → Artifact Version Integrity →

Configuration Change Control → Build Reproducibility & Environment

BREM governs:

Build Reproducibility & Environment

It is the fifth Core Module of SCFS.

Its principal upstream references are:

Governed Configuration Baseline State

from:

CBM — Configuration Baseline Module

Governed Software Build State

from:

BMM — Build Management Module

Governed Artifact Version Integrity State

from:

AVIM — Artifact Version Integrity Module

Governed Configuration Change State

from:

CCCM — Configuration Change Control Module

Its principal output is:

Governed Build Reproducibility & Environment State

This output contributes to the parent SCFS output:

Governed Software Configuration State

BREM may be invoked whenever build environments are established, modified, reconstructed, compared, retired, or reassessed, and whenever a material build reproducibility claim requires competent evaluation.

• Governed Space

Software Build Reproducibility & Environment Governance

This space governs the conditions under which software build environments

and reproducibility claims remain identifiable, evidence-supported, and reconstructable.

Its principal dimensions include:

• Reproducibility governance object identity

• Reproducibility purpose

• Reproducibility scope

• Reproducibility classification

• Original build identity

• Original build output reference

• Configuration baseline reference

• Source input identity

• Dependency input identity

• Build toolchain identity

• Build parameter identity

• Build environment identity

• Operating system conditions

• Runtime conditions

• Platform architecture

• Environment variables

• External build services

• Resource conditions

• Time-dependent build inputs

• Non-deterministic build conditions

• Environment isolation

• Environment specification

• Environment preservation

• Environment reconstruction

• Environment equivalence

• Reproduction authority

• Reproduction attempt identity

• Reproduction execution evidence

• Original–reproduced output comparison

• Reproducibility criteria

• Reproducibility verification

• Reproducibility evidence

• Evidence competence

• Evidence integrity

• Environment drift

• Reproducibility uncertainty

• Reproducibility conflicts

• Reproducibility gaps

• Critical reproducibility gaps

• Reproducibility limitations

• Reproducibility continuity

• Reproducibility reassessment

• Reproducibility governance states

The governed space is independent of any particular operating system, build platform, container technology, compiler, package manager, repository, or software delivery infrastructure.

Build Reproducibility Governance Object

Every material BREM assessment shall establish a Build Reproducibility

Governance Object.

The object represents an identifiable reproducibility claim, environment

reference, or reproduction assessment.

It should identify:

Reproducibility Identity: The identifiable assessment.

Reproducibility Purpose: The reason reproducibility is required.

Reproducibility Scope: The boundaries of the claim.

Original Build: The build being assessed.

Original Artifact: The relevant output artifact.

Configuration Baseline: The applicable configuration reference.

Build Inputs: The material source and dependency inputs.

Build Toolchain: The relevant tools and versions.

Build Parameters: The material execution settings.

Original Environment: The environment in which the original build

occurred.

Reconstruction Conditions: The conditions required to reproduce the build.

Reproduction Attempt: The subsequent build execution.

Comparison Criterion: The basis for evaluating reproduced output.

Reproducibility Evidence: Evidence supporting the claim.

Governance State: The recognized reproducibility condition.

The governance object shall remain distinguishable from a build script,

container image, environment manifest, or successful repeated build.

Reproducibility Governance Object ≠ Build Environment Manifest.

Reproducibility Purpose Governance

BREM shall identify the purpose of the reproducibility assessment.

Purposes may include:

• Build verification

• Artifact provenance support

• Release preparation

• Independent reconstruction

• Incident investigation

• Software maintenance

• Long-term preservation

• Supply-chain assurance

• Recovery

• Regulatory or contractual evidence support

Purpose shall remain associated with the required level of reproducibility.

Reproducibility Purpose ≠ Demonstrated Reproducibility.

Reproducibility Scope Governance

BREM shall establish the boundaries of the reproducibility claim.

Scope may concern:

• A single artifact

• A complete software package

• A container image

• A build stage

• A complete build pipeline

• A particular platform

• A defined toolchain

• A specified environment

• A defined historical build

Scope shall distinguish included, excluded, and unresolved material

conditions.

Defined Reproducibility Scope ≠ Complete Environment Knowledge.

Reproducibility Classification

BREM shall recognize the intended classification of the reproducibility

claim.

Relevant classifications may include:

• Exact artifact reproduction

• Byte-for-byte reproduction

• Deterministic build reproduction

• Environment-equivalent reproduction

• Functionally equivalent output reproduction

• Partial build-stage reproduction

• Historical build reconstruction

Classification shall remain distinct from the evidence supporting the claim.

Reproducibility Classification ≠ Verified Reproducibility State.

Original Build Identification

BREM shall identify the original build against which reproduction is assessed.

The reference may include:

• Build identity

• Build purpose

• Build inputs

• Toolchain references

• Execution environment

• Execution time

• Build output references

• Available build evidence

Original build execution governance remains within BMM.

Original Build Record ≠ Reconstructable Original Build.

Original Artifact Reference

BREM shall identify the material output artifact associated with the

original build.

The reference may include:

• Artifact identity

• Artifact version

• Content digest

• Package composition reference

• Relevant build correspondence

• Available integrity evidence

Artifact identity and version integrity remain governed by AVIM.

Original Artifact Reference ≠ Verified Original Artifact Content.

Configuration Baseline Reference

BREM shall associate the reproducibility assessment with the applicable

configuration baseline.

The baseline reference may identify:

• Relevant software configuration

• Component references

• Version references

• Build settings

• Environmental assumptions

• Applicable constraints

Baseline establishment remains governed by CBM.

Referenced Configuration Baseline ≠ Reconstructed Build Environment.

Source Input Identity

BREM shall establish material source input identity relevant to

reproducibility.

References may include:

• Source revision

• Source snapshot

• Generated source

• Build scripts

• Resource files

• Source transformation inputs

A source revision label alone may be insufficient where material generated or external inputs are involved.

Identical Source Revision ≠ Identical Build Inputs.

Dependency Input Identity

BREM shall identify material dependencies used in the original build and

required for reproduction.

Dependencies may include:

• Software libraries

• Packages

• Build plugins

• Code generators

• Runtime components

• External build resources

Dependency identity and suitability remain governed by SDS.

BREM governs whether the relevant dependency inputs are sufficiently identifiable for reproduction.

Declared Dependency Version ≠ Verified Dependency Input Identity.

Build Toolchain Identity

BREM shall identify the material tools and versions influencing build

output.

Toolchain references may include:

• Compiler

• Linker

• Interpreter

• Package manager

• Build orchestrator

• Code generator

• Packaging tool

• Container builder

Tool identity shall be sufficiently precise for the reproducibility claim

being assessed.

Same Tool Name ≠ Same Toolchain State.

Build Parameter Identity

BREM shall identify material build parameters influencing reproducibility.

Parameters may include:

• Compiler flags

• Optimization settings

• Build targets

• Feature flags

• Platform targets

• Packaging options

• Dependency resolution settings

• Reproducibility-specific settings

Declared parameter values shall remain distinguishable from effective

values.

Recorded Build Parameters ≠ Verified Effective Build Parameters.

Build Environment Identity

BREM shall establish an identifiable reference for the original build

environment.

Environment identity may involve:

• Operating system

• Runtime versions

• Platform architecture

• Toolchain

• Environment variables

• Build container

• External services

• Material resource conditions

An environment label shall not independently establish its actual

configuration.

Environment Name ≠ Verified Environment Identity.

Operating System Conditions

BREM shall identify material operating system conditions affecting

reproducibility.

Relevant conditions may include:

• Operating system distribution

• Operating system version

• Kernel characteristics

• System libraries

• Filesystem behavior

• Locale

• Timezone

• Relevant system configuration

Operating system identity shall be assessed at the level required by the

reproducibility claim.

Same Operating System Family ≠ Equivalent Build Environment.

Runtime Conditions

BREM shall identify material runtime conditions influencing build execution.

Runtime conditions may include:

• Interpreter versions

• Virtual machine versions

• Runtime libraries

• Language toolchains

• Execution frameworks

• Relevant environment settings

Runtime version references shall remain associated with the original and

reproduced build environments.

Same Runtime Name ≠ Same Runtime Conditions.

Platform Architecture

BREM shall identify material platform architecture conditions.

Relevant dimensions may include:

• Processor architecture

• Instruction set

• Operating system architecture

• Container architecture

• Target platform

• Cross-compilation conditions

Platform differences shall be evaluated relative to the defined

reproducibility criterion.

Same Source Code ≠ Platform-Independent Build Output.

Environment Variable Governance

BREM shall identify material environment variables affecting build

execution.

Variables may influence:

• Build paths

• Tool selection

• Dependency resolution

• Compiler behavior

• Feature activation

• Resource access

• Generated metadata

Sensitive values shall be protected through appropriate controls while preserving sufficient evidence of their material effect.

Environment Variable Declaration ≠ Effective Environment Variable State.

External Build Service Dependencies

BREM shall identify material external services influencing build execution.

Services may include:

• Package registries

• Remote compilers

• Code generation services

• Artifact repositories

• External build APIs

• Licensing services

• Hosted build infrastructure

External service availability does not establish reproducibility of the service's behavior.

External Service Availability ≠ Reproducible External Service Output.

Build Resource Conditions

BREM shall recognize material resource conditions affecting build execution.

Conditions may include:

• Memory availability

• Processor resources

• Storage availability

• Parallel execution capacity

• Network access

• Resource limits

Resource conditions shall be evaluated where they materially influence the reproducibility claim.

Sufficient Build Resources ≠ Equivalent Build Conditions.

Time-Dependent Build Inputs

BREM shall identify material build inputs that vary with time.

Examples may include:

• Build timestamps

• Dynamic version metadata

• Time-dependent generated files

• Expiring external resources

• Time-sensitive dependency resolution

• Date-dependent configuration

Time-dependent inputs shall remain explicit where they affect output correspondence.

Same Build Command ≠ Same Time-Dependent Inputs.

Non-Deterministic Build Conditions

BREM shall recognize material sources of non-determinism.

Sources may include:

• Randomized identifiers

• Uncontrolled timestamps

• Parallel execution ordering

• Unstable file ordering

• Dynamic dependency resolution

• Non-deterministic code generation

• External service variability

Non-determinism shall be assessed relative to the claimed reproducibility

level.

Repeated Successful Builds ≠ Deterministic Build Behavior.

Environment Isolation

BREM shall identify whether the build environment is sufficiently isolated for the relevant reproducibility claim.

Isolation mechanisms may include:

• Containers

• Virtual machines

• Sandboxed builds

• Dedicated build hosts

• Hermetic build systems

• Controlled dependency stores

Isolation shall not independently establish completeness or reproducibility.

Containerized Build ≠ Reproducible Build.

Environment Specification

BREM shall preserve material specifications describing the required build

environment.

Specifications may include:

• Environment manifests

• Container definitions

• Toolchain lockfiles

• Dependency lockfiles

• Build configuration records

• Platform requirements

• External service references

A complete-looking specification may omit material effective conditions.

Environment Specification ≠ Verified Environment State.

Environment Preservation

BREM shall support preservation of material build environment information.

Preservation may involve:

• Environment snapshots

• Toolchain records

• Dependency archives

• Container image references

• Build parameter records

• External service specifications

• Relevant historical evidence

Preservation shall distinguish stored references from independently

retrievable resources.

Preserved Environment Reference ≠ Recoverable Environment.

Environment Reconstruction

BREM shall govern the reconstruction of material build environment

conditions.

Reconstruction may involve:

• Restoring toolchains

• Re-establishing dependency versions

• Recreating configuration parameters

• Reconstructing runtime conditions

• Re-establishing build isolation

• Restoring required external inputs

Reconstruction shall preserve uncertainty where historical conditions cannot be sufficiently established.

Reconstructed Environment ≠ Verified Historical Environment.

Environment Equivalence

BREM shall assess whether original and reconstructed environments are equivalent for the defined reproducibility purpose.

Equivalence may concern:

• Identical toolchain versions

• Equivalent operating system behavior

• Equivalent dependency content

• Equivalent build parameters

• Equivalent execution conditions

• Equivalent external service outputs

Equivalence shall be evaluated against explicit criteria.

Environment Similarity ≠ Demonstrated Environment Equivalence.

Reproduction Authority

BREM shall identify the legitimate basis for performing material

reproduction activities.

Authority may concern:

• Access to source inputs

• Access to build environments

• Use of protected dependencies

• Execution of build tools

• Access to historical artifacts

• Use of external services

Authority shall remain bounded by permitted resources and actions.

Reproduction Capability ≠ Reproduction Authority.

Reproduction Attempt Identity

BREM shall establish an identifiable governance reference for each material

reproduction attempt.

The attempt should identify:

• Reproduction objective

• Original build reference

• Reproduction environment

• Relevant inputs

• Executing actor

• Execution time

• Resulting outputs

• Comparison evidence

Separate reproduction attempts shall remain distinguishable.

Repeated Reproduction Attempt ≠ Same Reproduction Event.

Reproduction Execution Evidence

BREM shall preserve competent evidence of material reproduction execution.

Evidence may include:

• Build execution records

• Input resolution records

• Toolchain observations

• Environment observations

• Effective parameter records

• Output generation records

• Execution deviations

BREM uses this evidence to assess reproducibility.

The general build execution process remains governed by BMM.

Reproduction Execution Log ≠ Demonstrated Reproducibility.

Reproducibility Comparison Criteria

BREM shall establish explicit criteria for comparing original and

reproduced outputs.

Criteria may include:

• Byte-for-byte identity

• Matching cryptographic digests

• Equivalent package composition

• Equivalent generated metadata

• Defined structural correspondence

• Defined functional equivalence

The selected criterion shall match the claim being evaluated.

Functional equivalence claims may require competent software quality assessment.

Output Similarity ≠ Defined Reproducibility Correspondence.

Original–Reproduced Output Comparison

BREM shall evaluate the relationship between original and reproduced

outputs.

Comparison may consider:

• Artifact identity

• Content digests

• Package composition

• Generated files

• Metadata differences

• Material output structure

• Relevant behavioral evidence

Differences shall be evaluated relative to the declared reproducibility

criterion.

Two Successful Build Outputs ≠ Reproduced Build Result.

Exact Reproducibility

BREM shall recognize exact reproducibility where competent evidence establishes that the reproduced output satisfies an explicitly defined exact-match criterion.

Exact matching may require:

• Identical material inputs

• Controlled execution conditions

• Deterministic transformations

• Identical resulting artifact content

• Competent comparison evidence

Exact reproducibility shall remain bounded by the artifacts and conditions

assessed.

Exact Artifact Match ≠ Complete Software Correctness.

Conditional Reproducibility

BREM shall recognize reproducibility that depends on explicit conditions.

Conditions may include:

• Specific toolchain versions

• Particular operating systems

• Defined dependency sources

• Controlled timestamps

• Restricted build parameters

• Particular external service behavior

Conditional reproducibility shall not be represented as universal

reproducibility.

Conditionally Reproducible Build ≠ Universally Reproducible Build.

Partial Reproducibility

BREM shall recognize where only part of a build can be reproduced.

Partial reproducibility may concern:

• Specific build stages

• Individual artifacts

• Defined platform targets

• Particular environment conditions

• Selected software components

Partial reproduction evidence shall not be generalized to the complete

build.

Partial Build Reproduction ≠ Complete Build Reproducibility.

Reproducibility Verification

BREM shall determine whether the claimed reproducibility level is sufficiently supported.

Verification may consider:

• Original build identity

• Original artifact identity

• Input correspondence

• Toolchain correspondence

• Environment equivalence

• Effective parameters

• Reproduction execution

• Output comparison

• Material uncertainty

Verification shall remain bounded by the declared scope and comparison

criterion.

Verified Reproducibility ≠ Verified Software Quality.

Reproducibility Evidence

BREM shall preserve evidence supporting material reproducibility claims.

Evidence may include:

• Original build records

• Environment specifications

• Toolchain records

• Dependency references

• Build parameter records

• Reproduction execution records

• Artifact comparison results

• Integrity verification records

Evidence shall remain associated with the specific reproducibility claim it

supports.

Reproducibility Evidence Presence ≠ Reproducibility Claim Validity.

Evidence Competence

BREM shall assess whether evidence is competent to support the relevant reproducibility determination.

For example:

An environment manifest may support declared environment composition.

A toolchain record may support tool version identification.

An execution log may support observed reproduction activity.

A cryptographic digest comparison may support byte-level output correspondence.

These records establish different facts.

Evidence of Environment Specification ≠ Evidence of Environment

Equivalence.

Evidence Integrity

BREM shall preserve the integrity of material reproducibility evidence.

Relevant considerations may include:

• Evidence source

• Content integrity

• Temporal relevance

• Original build correspondence

• Reproduction attempt correspondence

• Artifact identity correspondence

• Material contradictions

• Preservation history

An intact reproduction record does not independently establish that the reproduction claim is valid.

Intact Reproduction Record ≠ Verified Reproducibility.

Environment Drift

BREM shall recognize material changes in build environments over time.

Drift may involve:

• Toolchain updates

• Dependency updates

• Operating system changes

• Runtime changes

• Environment variable changes

• External service changes

• Infrastructure configuration changes

Drift shall be evaluated relative to the reproducibility conditions

required.

Environment Drift ≠ Automatic Reproducibility Failure.

Reproducibility Deviation

A Reproducibility Deviation exists where material reproduction conditions differ from those required by the assessed claim.

Examples may include:

• Different compiler version

• Different dependency content

• Changed build parameters

• Different operating system behavior

• Missing external build inputs

• Uncontrolled timestamps

• Changed code generation behavior

BREM shall preserve the deviation and its supporting evidence.

Successful Reproduction Attempt with Deviations ≠ Verified Exact

Reproducibility.

Reproducibility Uncertainty

BREM shall preserve material uncertainty concerning build environments or

reproducibility claims.

Uncertainty may arise where:

• Original environment conditions are unknown

• Historical dependencies are unavailable

• Toolchain identity is incomplete

• Effective parameters are unresolved

• External service behavior cannot be reconstructed

• Output comparison evidence is insufficient

Uncertainty shall not be silently converted into reproducibility acceptance.

Unresolved Reproducibility Conditions ≠ Verified Reproducibility

Conditions.

Reproducibility Conflict

A Reproducibility Conflict exists where material reproducibility claims or evidence are incompatible.

Examples may include:

• Conflicting environment records

• Contradictory toolchain references

• Inconsistent artifact comparison results

• Conflicting dependency identity records

• Incompatible reproduction outcomes

BREM shall preserve the conflict and its supporting evidence.

Conflicting Reproducibility Evidence ≠ Permission to Select the Preferred

Result.

Reproducibility Gap

A Reproducibility Gap exists where material information required for competent reproducibility assessment remains insufficiently established.

Examples may include:

• Missing original toolchain identity

• Unknown effective build parameters

• Unavailable dependency content

• Missing environment records

• Unresolved external build inputs

• Missing output comparison evidence

BREM shall identify the gap and its material significance.

Reproducibility Evidence Gap ≠ Proven Non-Reproducibility.

Critical Reproducibility Gap

A Critical Reproducibility Gap exists where missing information prevents competent assessment of a materially consequential build reproducibility claim.

Examples may include:

- A critical production artifact whose original build environment cannot

be reconstructed. - A long-term software maintenance obligation without preserved build toolchain information. - A release candidate whose output cannot be independently reproduced under defined conditions. - A recovery build dependent on unavailable historical external services.

BREM shall preserve the critical gap for competent downstream assessment.

It does not independently determine software shutdown, security compromise, or legal liability.

Critical Reproducibility Gap ≠ Automatic Software Failure.

Reproducibility Failure

A Reproducibility Failure exists where competent evidence establishes that the assessed reproducibility criterion is not satisfied under the defined conditions.

Examples may include:

• Verified output digest mismatch under an exact-match criterion

• Demonstrated inability to reproduce required build stages under the

• Confirmed non-deterministic output violating the declared

• Established environment incompatibility preventing the required

specified conditions. reproducibility requirement. reproduction.

BREM shall distinguish established failure from missing evidence or

incomplete reproduction attempts.

Verified Reproducibility Failure ≠ Mere Reproducibility Uncertainty.

Reproducibility Limitations

BREM shall preserve explicit limitations affecting reproducibility claims.

Limitations may include:

• Platform specificity

• Toolchain restrictions

• Dependency availability

• External service requirements

• Historical environment constraints

• Partial artifact coverage

• Defined comparison criteria

• Known non-deterministic conditions

Limitations shall remain associated with the reproducibility state.

Bounded Reproducibility ≠ Unrestricted Reproducibility.

Reproducibility Continuity

BREM shall preserve material reproducibility relationships across software

lifecycle transitions.

Continuity may involve:

• Original build references

• Historical environment records

• Toolchain preservation

• Dependency preservation

• Reproduction attempts

• Artifact comparison history

• Environment changes

• Reproducibility reassessments

Continuity shall preserve the distinction between historical

reproducibility and current reproducibility capability.

Historical Reproducibility ≠ Current Reproducibility Capability.

Reproducibility Reassessment

BREM shall support reassessment when material environment or reproducibility conditions change.

Triggers may include:

• Toolchain changes

• Dependency changes

• Environment drift

• New artifact evidence

• Reproduction failures

• New verification methods

• Historical resource loss

• Changed reproducibility requirements

• Conflicting reproduction results

Reassessment shall preserve relevant historical determinations and evidence.

Reproducibility Reassessment ≠ Historical Reproducibility Record Erasure.

Reproducibility Governance States

BREM shall support identifiable governance states.

Possible states include:

Identified: A reproducibility governance object has been recognized.

Specified: Reproducibility requirements and comparison criteria have been

defined.

Environment-Recorded: Material environment information has been recorded.

Environment-Unresolved: Material environment conditions remain

insufficiently established.

Reconstruction-Pending: Required environment reconstruction has not been

completed.

Reconstructed: A reproduction environment has been established.

Equivalence-Unresolved: Environment equivalence remains insufficiently

established.

Reproduction-In-Progress: A reproduction attempt is underway.

Reproduction-Completed: The defined reproduction attempt has completed.

Conditionally Reproducible: Reproducibility is supported under explicit conditions.

Partially Reproducible: Reproducibility is supported only within a limited scope.

Reproducibility-Verified: The defined reproducibility criterion has been

sufficiently demonstrated.

Reproducibility-Failed: Competent evidence establishes failure of the

defined criterion.

Deviation-Detected: Material differences from required reproduction

conditions have been identified.

Evidence-Limited: Available evidence is insufficient for the required determination.

Conflicted: Material reproducibility claims or evidence are contradictory.

Historically Reproducible: Historical evidence supports reproducibility

under previously established conditions.

Currently Unreconstructable: Required current reconstruction cannot be

completed under the assessed conditions.

Superseded: A later governance state has replaced the earlier state.

Unresolved: Available information does not support a sufficiently determined state.

These states shall not independently establish software quality, security,

release readiness, or deployment authorization.

Reproducibility-Verified ≠ Software Product Validated.

Governed Build Reproducibility & Environment State

The principal output of BREM is:

Governed Build Reproducibility & Environment State

The state shall preserve sufficient information to identify:

Reproducibility Governance Object: The reproducibility assessment.

Reproducibility Identity: The identifiable claim or assessment.

Reproducibility Purpose: The reason reproducibility is required.

Reproducibility Scope: The boundaries of the claim.

Reproducibility Classification: The intended level or type.

Original Build: The build being assessed.

Original Artifact: The relevant original output.

Configuration Baseline: The applicable baseline reference.

Source Inputs: Material source input identity.

Dependency Inputs: Material dependency identity.

Build Toolchain: Relevant tools and versions.

Build Parameters: Material execution parameters.

Original Environment: The environment in which the original build

occurred.

Operating System Conditions: Relevant system characteristics.

Runtime Conditions: Material runtime characteristics.

Platform Architecture: Relevant platform conditions.

Environment Variables: Material effective settings.

External Build Services: Relevant external execution dependencies.

Resource Conditions: Material resource constraints.

Time-Dependent Inputs: Relevant temporal influences.

Non-Deterministic Conditions: Material sources of output variability.

Environment Isolation: Relevant isolation mechanisms.

Environment Specification: The declared environment requirements.

Environment Preservation: Available preserved environment evidence.

Environment Reconstruction: The reconstructed environment state.

Environment Equivalence: The assessed relationship between original and

reconstructed environments.

Reproduction Authority: The legitimate basis for reproduction activities.

Reproduction Attempt: The identifiable subsequent build.

Reproduction Execution Evidence: Evidence of actual reproduction activity.

Comparison Criterion: The required output correspondence.

Output Comparison: The observed original–reproduced relationship.

Exact Reproducibility: The assessed exact-match condition.

Conditional Reproducibility: Relevant explicit reproducibility conditions.

Partial Reproducibility: Relevant scope limitations.

Reproducibility Verification: The result of competent reproducibility

assessment.

Reproducibility Evidence: Evidence supporting material claims.

Evidence Competence: The suitability of evidence for those claims.

Evidence Integrity: The condition of supporting evidence.

Environment Drift: Material environmental changes.

Reproducibility Deviations: Differences from required reproduction

conditions.

Reproducibility Uncertainty: Material unresolved conditions.

Reproducibility Conflicts: Contradictory claims or evidence.

Reproducibility Gaps: Missing material information.

Critical Reproducibility Gaps: Material gaps requiring competent

attention.

Reproducibility Failures: Established failures of defined criteria.

Reproducibility Limitations: Explicit scope and condition restrictions.

Reproducibility Continuity: Preservation of material historical

relationships.

Reproducibility Reassessment: Material reassessment history.

Reproducibility Governance State: The recognized reproducibility

condition.

Downstream References: Information required by competent SCFS and SLGA™

governance.

The state shall not independently establish software correctness, security,

release authorization, or deployment authorization.

Minimum Implementation Framework

A conforming implementation of BREM must provide the following six functional capabilities.

• Reproducibility Object, Purpose, Scope, Classification, and Original

Build Governance

Capability to establish identifiable reproducibility governance objects,

define purpose, scope, and classification, and associate the assessment with relevant original build, artifact, and configuration baseline references.

The implementation must distinguish reproducibility requirements from demonstrated reproducibility.

• Build Input, Toolchain, Parameter, and Environment Identity Governance

Capability to identify material source and dependency inputs, toolchain

versions, effective build parameters, operating system conditions, runtime characteristics, platform architecture, environment variables, external services, and resource conditions.

The implementation must distinguish recorded environment specifications from verified effective environment states.

• Environment Preservation, Reconstruction, Isolation, and Equivalence

Governance

Capability to preserve material environment references, recognize isolation

mechanisms, reconstruct relevant execution conditions, and assess equivalence between original and reconstructed environments.

The implementation must distinguish environment similarity from demonstrated equivalence and preserved references from recoverable resources.

• Reproduction Execution, Output Comparison, and Verification Governance

Capability to identify reproduction attempts, preserve execution evidence,

define comparison criteria, assess original–reproduced output correspondence, and recognize exact, conditional, partial, or failed reproducibility.

The implementation must distinguish repeated successful builds from demonstrated reproduction of the required result.

• Reproducibility Evidence, Drift, Deviation, Gap, Conflict, and

Continuity Governance

Capability to associate reproducibility claims with competent evidence,

preserve evidence integrity, identify environment drift, execution deviations, uncertainty, evidence gaps, critical gaps, conflicts, established failures, limitations, and historical continuity.

The implementation must distinguish missing reproducibility evidence from proven non-reproducibility.

• Governed Build Reproducibility & Environment State and SCFS Handoff

Governance

Capability to produce a Governed Build Reproducibility & Environment State

with sufficient original build identity, input, toolchain, environment, reconstruction, equivalence, reproduction attempt, output comparison, verification, evidence, uncertainty, limitation, and continuity information for competent parent SCFS governance.

The implementation must not represent reproducibility as proof of software correctness, security, release readiness, or deployment authorization.

GIE™ Compatibility

BREM is compatible with GIE™ as a downstream methodological analysis and execution layer.

GIE™ may support:

• Reproducibility governance object identification

• Reproducibility purpose analysis

• Reproducibility scope assessment

• Reproducibility classification

• Original build reference analysis

• Original artifact reference analysis

• Configuration baseline correspondence analysis

• Source input identity analysis

- Dependency input identity analysis
- Toolchain identity assessment
- Build parameter assessment
- Environment identity analysis
- Operating system condition assessment
- Runtime condition assessment
- Platform architecture analysis
- Environment variable analysis
- External build service dependency analysis
- Build resource condition assessment
- Time-dependent input recognition
- Non-deterministic condition detection
- Environment isolation assessment
- Environment specification analysis
- Environment preservation analysis
- Environment reconstruction support
- Environment equivalence assessment
- Reproduction authority analysis
- Reproduction attempt identification
- Reproduction execution evidence analysis
- Reproducibility comparison criterion assessment
- Original–reproduced output comparison
- Exact reproducibility assessment
- Conditional reproducibility assessment
- Partial reproducibility assessment
- Reproducibility verification support
- Evidence competence assessment
- Evidence integrity assessment
- Environment drift detection
- Reproducibility deviation detection
- Reproducibility uncertainty recognition
- Reproducibility conflict detection
- Reproducibility gap detection
- Critical reproducibility gap detection
- Reproducibility failure recognition
- Reproducibility limitation analysis

• Reproducibility continuity mapping

• Reproducibility reassessment support

• Governed reproducibility and environment state representation

GIE™ may identify candidate environment relationships, detect possible reproducibility deviations, or propose reproducibility assessments.

Such outputs shall remain distinguishable from sufficiently evidenced

reproducibility determinations.

GIE™ must not fabricate historical environment records, invent successful reproduction results, infer environment equivalence from labels, or represent repeated successful builds as proof of reproducibility.

OOF® logic adapts to the governed reality. The engine adapts to OOF®

logic.

Governance Logic

The canonical BREM governance logic is:

Receive Governed Configuration Baseline State

↓

Receive Governed Software Build State

↓

Receive Governed Artifact Version Integrity State

↓

Receive Relevant Governed Configuration Change State

↓

Identify Build Reproducibility Governance Object

↓

Establish Reproducibility Purpose and Scope

↓

Determine Reproducibility Classification

↓

Identify Original Build and Artifact

↓

Identify Material Source and Dependency Inputs

↓

Identify Toolchain and Effective Build Parameters

↓

Establish Original Build Environment Identity

↓

Identify Operating System, Runtime, and Platform Conditions

↓

Identify Environment Variables and External Services

↓

Identify Resource and Time-Dependent Conditions

↓

Identify Non-Deterministic Build Conditions

↓

Assess Environment Isolation

↓

Preserve Environment Specification and Historical References

↓

Identify Reproduction Authority

↓

Establish Reconstruction Requirements

↓

Reconstruct Material Environment Conditions

↓

Assess Original–Reconstructed Environment Equivalence

↓

Identify Reproduction Attempt

↓

Observe Reproduction Execution

↓

Establish Output Comparison Criteria

↓

Compare Original and Reproduced Outputs

↓

Assess Exact, Conditional, or Partial Reproducibility

↓

Associate Reproducibility Evidence

↓

Assess Evidence Competence and Integrity

↓

Verify Material Reproducibility Conditions

↓

Identify Environment Drift and Reproduction Deviations

↓

Identify Reproducibility Uncertainty

↓

Identify Reproducibility Conflicts

↓

Identify Reproducibility Gaps and Critical Gaps

↓

Identify Established Reproducibility Failures

↓

Determine Reproducibility Governance State

↓

Establish Governed Build Reproducibility & Environment State

↓

Preserve Reproducibility Limitations and Continuity

↓

Handoff to Parent SCFS Governance

↓

Reassess When Material Environment or Reproducibility Conditions Change

The principal output is:

Governed Build Reproducibility & Environment State

Governance Outputs

BREM may produce:

Build Reproducibility Governance Object

Reproducibility Identity Record

Reproducibility Purpose Record

Reproducibility Scope Record

Reproducibility Classification Record

Original Build Reference

Original Artifact Reference

Configuration Baseline Reference

Source Input Identity Record

Dependency Input Identity Record

Build Toolchain Identity Record

Build Parameter Identity Record

Build Environment Identity Record

Operating System Condition Record

Runtime Condition Record

Platform Architecture Record

Environment Variable Record

External Build Service Dependency Record

Build Resource Condition Record

Time-Dependent Build Input Record

Non-Deterministic Build Condition Record

Environment Isolation Assessment

Environment Specification Record

Environment Preservation Record

Environment Reconstruction Record

Environment Equivalence Assessment

Reproduction Authority Record

Reproduction Attempt Record

Reproduction Execution Evidence Record

Reproducibility Comparison Criterion

Original–Reproduced Output Comparison Record

Exact Reproducibility Assessment

Conditional Reproducibility Assessment

Partial Reproducibility Assessment

Reproducibility Verification Record

Reproducibility Evidence Register

Evidence Competence Assessment

Evidence Integrity Assessment

Environment Drift Record

Reproducibility Deviation Record

Reproducibility Uncertainty State

Reproducibility Conflict

Reproducibility Gap

Critical Reproducibility Gap

Reproducibility Failure Record

Reproducibility Limitation Record

Reproducibility Continuity Record

Reproducibility Reassessment Record

Reproducibility Governance State

SCFS Reproducibility Handoff Record

The principal output is:

Governed Build Reproducibility & Environment State

Critical Distinctions

BREM preserves the following distinctions:

Repeated Build Execution ≠ Demonstrated Build Reproducibility

Reproducibility Governance Object ≠ Build Environment Manifest

Reproducibility Purpose ≠ Demonstrated Reproducibility

Defined Reproducibility Scope ≠ Complete Environment Knowledge

Reproducibility Classification ≠ Verified Reproducibility State

Original Build Record ≠ Reconstructable Original Build

Original Artifact Reference ≠ Verified Original Artifact Content

Referenced Configuration Baseline ≠ Reconstructed Build Environment

Identical Source Revision ≠ Identical Build Inputs

Declared Dependency Version ≠ Verified Dependency Input Identity

Same Tool Name ≠ Same Toolchain State

Recorded Build Parameters ≠ Verified Effective Build Parameters

Environment Name ≠ Verified Environment Identity

Same Operating System Family ≠ Equivalent Build Environment

Same Runtime Name ≠ Same Runtime Conditions

Same Source Code ≠ Platform-Independent Build Output

Environment Variable Declaration ≠ Effective Environment Variable State

External Service Availability ≠ Reproducible External Service Output

Sufficient Build Resources ≠ Equivalent Build Conditions

Same Build Command ≠ Same Time-Dependent Inputs

Repeated Successful Builds ≠ Deterministic Build Behavior

Containerized Build ≠ Reproducible Build

Environment Specification ≠ Verified Environment State

Preserved Environment Reference ≠ Recoverable Environment

Reconstructed Environment ≠ Verified Historical Environment

Environment Similarity ≠ Demonstrated Environment Equivalence

Reproduction Capability ≠ Reproduction Authority

Repeated Reproduction Attempt ≠ Same Reproduction Event

Reproduction Execution Log ≠ Demonstrated Reproducibility

Output Similarity ≠ Defined Reproducibility Correspondence

Two Successful Build Outputs ≠ Reproduced Build Result

Exact Artifact Match ≠ Complete Software Correctness

Conditionally Reproducible Build ≠ Universally Reproducible Build

Partial Build Reproduction ≠ Complete Build Reproducibility

Verified Reproducibility ≠ Verified Software Quality

Reproducibility Evidence Presence ≠ Reproducibility Claim Validity

Evidence of Environment Specification ≠ Evidence of Environment Equivalence

Intact Reproduction Record ≠ Verified Reproducibility

Environment Drift ≠ Automatic Reproducibility Failure

Successful Reproduction Attempt with Deviations ≠ Verified Exact Reproducibility

Unresolved Reproducibility Conditions ≠ Verified Reproducibility Conditions

Conflicting Reproducibility Evidence ≠ Permission to Select the Preferred Result

Reproducibility Evidence Gap ≠ Proven Non-Reproducibility

Critical Reproducibility Gap ≠ Automatic Software Failure

Verified Reproducibility Failure ≠ Mere Reproducibility Uncertainty

Bounded Reproducibility ≠ Unrestricted Reproducibility

Historical Reproducibility ≠ Current Reproducibility Capability

Reproducibility Reassessment ≠ Historical Reproducibility Record Erasure

Reproducibility-Verified ≠ Software Product Validated

Cross-Architecture Boundary

BREM governs software build environment identity, reconstruction,

equivalence, reproducibility verification, and lifecycle continuity within SCFS and SLGA™.

It does not independently establish configuration baselines, execute complete build-management processes, certify artifact provenance, authorize configuration changes, validate software quality, certify software security, authorize release, or authorize deployment.

Within SCFS:

CBM — Configuration Baseline Module governs configuration baseline

identity, authority, composition, applicability, evidence, and continuity.

BMM — Build Management Module governs build identity, inputs, tools,

parameters, authority, execution, observation, and resulting-artifact correspondence.

AVIM — Artifact Version Integrity Module governs artifact identity,

content identity, version attribution, version correspondence, integrity evidence, and lifecycle continuity.

CCCM — Configuration Change Control Module governs configuration change

identity, authority, scope, execution, resulting-state correspondence, verification, and continuity.

BREM — Build Reproducibility & Environment Module governs build

environment identity, preservation, reconstruction, equivalence, reproducibility criteria, output comparison, verification, and continuity.

Within SLGA™:

Software Requirements Standard (SRS) governs software requirements.

Software Creation Standard (SCS) governs software creation.

Software Dependency Standard (SDS) governs software dependencies.

Software Quality Standard (SQS) governs software quality.

Software Security Standard (SSS) governs software security.

Software Release Standard (SRLS) governs software release.

Software Deployment Standard (SDPS) governs software deployment.

Software Change Management Standard (SCMS) governs broader lifecycle

change management.

Software Retirement Standard (SRTS) governs software retirement.

Software Lifecycle Control Standard (SLCS) governs cross-cutting

lifecycle control.

Relevant interoperating OOF® architectures include:

GOA™ — Governance Architecture: General authority and delegation

governance.

AGA™ — Accountability Governance Architecture: Relevant responsibility

and accountability relationships.

ORA™ — Operational Reality Architecture: Competent assessment of actual

build and environment conditions.

ASGA™ — Autonomous Systems Governance Architecture: Relevant autonomous

reproduction actions and operational boundaries.

AGI™ — Agent Governance Infrastructure: Relevant agent permissions and

tool-path governance.

BREM preserves build-reproducibility-specific governance without replacing

these competent domains.

Canonical Boundary

BREM begins when a material software build environment requires identification, preservation, reconstruction, equivalence assessment, or reproducibility verification.

BREM governs:

• Reproducibility governance objects

• Reproducibility purpose

• Reproducibility scope

• Reproducibility classification

• Original build identity

• Original artifact references

- Configuration baseline references
- Source input identity
- Dependency input identity
- Toolchain identity
- Build parameter identity
- Build environment identity
- Operating system conditions
- Runtime conditions
- Platform architecture
- Environment variables
- External build services
- Resource conditions
- Time-dependent inputs
- Non-deterministic build conditions
- Environment isolation
- Environment specification
- Environment preservation
- Environment reconstruction
- Environment equivalence
- Reproduction authority
- Reproduction attempt identity
- Reproduction execution evidence
- Comparison criteria
- Original–reproduced output comparison
- Exact reproducibility
- Conditional reproducibility
- Partial reproducibility
- Reproducibility verification
- Reproducibility evidence
- Evidence competence
- Evidence integrity
- Environment drift
- Reproducibility deviations
- Reproducibility uncertainty
- Reproducibility conflicts
- Reproducibility gaps

• Critical reproducibility gaps

• Reproducibility failures

• Reproducibility limitations

• Reproducibility continuity

• Reproducibility reassessment

• Reproducibility governance states

BREM does not independently determine:

• Software requirements validity

• Software creation authorization

• Complete dependency suitability

• Configuration baseline establishment

• Complete build execution governance

• Artifact-version integrity certification

• Configuration change authorization

• Software quality

• Software security

• Release authorization

• Deployment authorization

• Legal compliance

• Legal liability

BREM ends when the material environment and reproducibility conditions have been represented as a:

Governed Build Reproducibility & Environment State

with sufficient original build identity, inputs, toolchain, environment,

reconstruction, equivalence, reproduction attempt, comparison, verification, evidence, uncertainty, limitation, and continuity information for competent parent SCFS governance.

Use Case 1 — Two Builds Use Identical Source Code but Produce Different

Artifacts

An organization maintains a software application through an automated build pipeline.

The application source code remains unchanged between two builds.

Both builds execute successfully.

However, the resulting artifacts have different cryptographic digests.

The organization initially assumes that the builds should be identical because they used the same source revision.

A reproducibility assessment identifies that:

• The compiler version changed

• A dependency was resolved dynamically

• The build process embedded timestamps

• The second build used different optimization settings

BREM evaluates:

Original Build → Source Inputs → Dependency Inputs → Toolchain →

Parameters → Environment → Reproduction Attempt → Output Comparison

The assessment identifies:

• The same declared source revision

• Material differences in build conditions

• Different resulting artifact content

• Insufficient support for exact reproducibility

• Specific conditions requiring control before a new exact-reproduction

claim can be assessed.

The relevant governance state may be:

Deviation-Detected

or:

Reproducibility-Failed

where the defined exact-match criterion was required and demonstrably not satisfied.

The critical distinctions are:

Identical Source Revision ≠ Identical Build Inputs.

Same Tool Name ≠ Same Toolchain State.

Repeated Successful Builds ≠ Deterministic Build Behavior.

Two Successful Build Outputs ≠ Reproduced Build Result.

The result: BREM identifies the environmental and execution conditions

that prevent a successful repeated build from establishing exact artifact reproducibility.

Use Case 2 — A Critical Software Product Cannot Be Rebuilt Five Years

Later

An organization operates a critical software product that must remain maintainable for many years.

The original build was completed successfully.

The organization preserved:

• Source code

• A build script

• A general operating system reference

• An application version label

However, the organization did not preserve:

• The exact compiler version

• Several historical dependencies

• A proprietary code generation tool

• Material build environment variables

• An external build service specification

Five years later, the organization must rebuild the software to correct a defect.

The source code remains available.

The build script still exists.

But the original build environment cannot be reconstructed with sufficient

confidence.

BREM evaluates:

Original Build → Historical Environment → Toolchain Preservation →

Dependency Availability → Reconstruction → Reproduction Evidence

The assessment identifies:

• An identifiable historical software build

• Incomplete historical environment evidence

• Unavailable material build dependencies

• An unrecoverable external tool

• Insufficient evidence to reconstruct the original build conditions

• No demonstrated reproduction of the original artifact

The relevant governance state may be:

Currently Unreconstructable

with a:

Critical Reproducibility Gap

where the missing conditions prevent competent assessment of a critical maintenance obligation.

The critical distinctions are:

Original Build Record ≠ Reconstructable Original Build.

Preserved Environment Reference ≠ Recoverable Environment.

Reconstructed Environment ≠ Verified Historical Environment.

Historical Reproducibility ≠ Current Reproducibility Capability.

The result: BREM reveals that long-term software maintainability depends not merely on preserving source code, but on preserving sufficient build-environment identity, dependencies, tools, conditions, and evidence to support future reconstruction.

Canonical Closing Statement

Software cannot be considered reproducible merely because its source code

has been preserved.

A build script may remain available while the toolchain required to execute it has disappeared.

A container image may exist while its external dependencies have changed.

A build may complete successfully under new environmental conditions while producing a materially different artifact.

And two builds may generate identical-looking outputs without sufficient evidence that the required reproducibility conditions were satisfied.

Repeated Build Execution ≠ Demonstrated Build Reproducibility.

Identical Source Revision ≠ Identical Build Inputs.

Environment Specification ≠ Verified Environment State.

Environment Similarity ≠ Demonstrated Environment Equivalence.

Two Successful Build Outputs ≠ Reproduced Build Result.

Historical Reproducibility ≠ Current Reproducibility Capability.

These distinctions become increasingly important as software development

depends on complex toolchains, external package registries, hosted build infrastructure, generated code, containerized environments, and autonomous software development systems.

An AI agent may successfully rebuild an application.

An automated pipeline may report completion.

A container system may recreate an environment.

None of these actions independently establishes that the original build conditions were sufficiently reconstructed or that the reproduced output satisfies the required correspondence criterion.

BREM establishes the methodological requirement that reproducibility remain an evidence-supported relationship between an original build, its material execution conditions, a reconstructed environment, a subsequent build, and the outputs being compared.

It does not require every software build to be byte-for-byte reproducible.

It requires reproducibility claims to identify their scope, conditions,

comparison criteria, evidence, and limitations.

It also recognizes that reproducibility is a lifecycle capability.

A build that can be reproduced today may become unreconstructable tomorrow if material tools, dependencies, services, or environment records are lost.

For this reason, environment preservation and historical continuity are not merely technical conveniences.

They are material conditions of long-term software governance.

BREM transforms software build reproducibility from an assumption based on

repeated execution into a governed, evidence-supported relationship between build inputs, environmental conditions, reconstruction processes, and resulting artifacts — ensuring that software organizations can establish not merely whether a build ran again, but whether the original result can be meaningfully reproduced under sufficiently defined and verifiable conditions.

A software build is not reproducible because its source code exists, its build script runs, or a new artifact is generated. It is reproducible within a defined scope when the material inputs, toolchain, environment, execution conditions, comparison criteria, and supporting evidence sufficiently establish the relationship between the original build and the reproduced result.